TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

A

Project Report

On

Automated CI/CD Pipeline with Containerised Web Application and Monitoring using Prometheus and Grafana

Submitted By:

Shreya Uprety (PUL078BCT086)

Submitted To:

Sanjaya Subedi

Department of Electronics & Computer Engineering

Pulchowk Campus, Institute of Engineering

Tribhuvan University

May, 2026

Acknowledgements

I would like to express my sincere gratitude to my supervisor, **Sanjaya Subedi**, for his invaluable guidance, continuous encouragement, and constructive feedback throughout the duration of this project. His expertise in the field of software engineering and system design has been instrumental in shaping the direction and quality of this work.

I am also grateful to the faculty members of the Department of Electronics and Computer Engineering at Pulchowk Campus for providing a solid academic foundation that made this project possible.

I extend my appreciation to the open-source communities behind Jenkins, Docker, Prometheus, Grafana, and Ansible, whose freely available documentation and tools form the technical backbone of this project.

Finally, I am deeply thankful to my family and friends for their unwavering support and encouragement throughout my studies.

Shreya Uprety
Department of Electronics and Computer Engineering
Pulchowk Campus, Tribhuvan University
May 13, 2026

Abstract

Modern software development demands rapid, reliable, and repeatable delivery of applications. This project presents the design and implementation of a complete **Automated CI/CD (Continuous Integration/Continuous Deployment) pipeline** for a containerised Node.js web application, incorporating real-time monitoring through Prometheus and Grafana.

The system integrates **Jenkins** as the orchestration engine, **Docker** for containerisation, **Docker Hub** as the image registry, and **Ansible** for idempotent configuration management and deployment. The pipeline is triggered automatically via GitHub webhooks on every push to the `main` branch, executing stages for dependency installation, automated testing (Jest), Docker image building and pushing, and finally Ansible-driven deployment to a remote server.

A monitoring stack composed of Prometheus scraping application metrics every 15 seconds, and Grafana providing pre-provisioned dashboards, gives real-time visibility into HTTP request rates, Node.js heap memory usage, and request latency percentiles.

The implementation demonstrates that *pipeline-as-code* (Jenkinsfile), *infrastructure-as-code* (Ansible playbooks), and container-based delivery significantly reduce manual intervention, improve deployment consistency, and enable faster feedback loops. Test failures abort the pipeline before deployment, preventing broken code from reaching production.

Keywords: CI/CD, Jenkins, Docker, Ansible, Prometheus, Grafana, DevOps, Containerisation, Node.js, Automation.

Contents

1 Introduction **8**

1.1 Background 8

1.2 Problem Statement 8

1.3 Objectives 8

1.4 Scope 9

1.5 Report Organisation 9

2 Literature Review **10**

2.1 CI/CD Tools 10

2.1.1 Jenkins 10

2.1.2 GitHub Actions 10

2.1.3 GitLab CI/CD 10

2.1.4 Comparison Summary 11

2.2 Containerisation: Docker vs Virtual Machines 11

2.3 Infrastructure as Code Tools 11

2.4 Monitoring: Prometheus and Grafana 12

3 Requirement Analysis **13**

3.1 Functional Requirements 13

3.2 Non-Functional Requirements 14

3.3 Hardware and Software Requirements 14

4 System Design and Methodology **16**

4.1 Overall Architecture 16

4.2 Jenkins Pipeline Design 16

4.3 Ansible Deployment Flow 17

4.4 Monitoring Stack Design 18

5 Implementation **19**

5.1 Running Application 19

5.2 Application Source (app/index.js) 20

5.3 Dockerfile 21

5.4 Jenkins CI/CD Pipeline 21
5.5 Ansible Playbook 23
5.6 Prometheus Configuration 24
5.7 Grafana Dashboard 24
5.8 Jest Test Suite 24

6 Results and Discussion 26
6.1 Pipeline Execution 26
6.1.1 Successful Build 26
6.1.2 Failed Build (Test Failure) 26
6.2 Prometheus Scraping Results 27
6.3 Grafana Dashboard Observations 29
6.4 Discussion 31

7 Conclusion 32
7.1 Summary 32
7.2 Limitations 32
7.3 Future Work 33

List of Figures

4.1 System Architecture Overview 16
4.2 Jenkins Pipeline Stage Flow 17

5.1 To-Do API root endpoint (`GET /`) returning the todo list as JSON 19
5.2 Health check endpoint (`GET /health`) confirming application uptime . . 20
5.3 Jenkins dashboard showing the devops-cicd-pipeline job 22
5.4 Pipeline job page showing build history and stage results 22

6.1 Prometheus targets page showing both scrape targets in UP state 27
6.2 Prometheus expression browser: `http_requests_total` with per-route and per-status-code labels 28
6.3 Prometheus graph: per-route HTTP request rate (req/s) over 15 minutes 29
6.4 Grafana dashboard showing HTTP request rate, heap memory, request duration, and total requests 30

List of Tables

2.1 CI/CD Tool Comparison 11
2.2 Docker Containers vs Virtual Machines 11

3.1 Hardware Requirements 14
3.2 Software Requirements 15

6.1 Typical Pipeline Stage Durations 26

List of Abbreviations

CI	Continuous Integration
CD	Continuous Deployment / Continuous Delivery
VCS	Version Control System
IaC	Infrastructure as Code
SCM	Source Code Management
VM	Virtual Machine
API	Application Programming Interface
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
YAML	YAML Ain't Markup Language
CPU	Central Processing Unit
RAM	Random Access Memory
OS	Operating System
SLA	Service Level Agreement
MTTR	Mean Time to Recovery
p95	95th Percentile
SSH	Secure Shell
IoE	Institute of Engineering
ECE	Electronics and Computer Engineering

Chapter 1

Introduction

1.1 Background

The software industry has undergone a paradigm shift from monolithic, infrequent releases towards continuous delivery of small, testable increments. **DevOps**, a cultural and technical movement that bridges development and operations, has emerged as the dominant strategy for achieving this agility [8]. At its core, a CI/CD pipeline automates the path from a developer's code commit to a running production deployment, eliminating error-prone manual processes.

1.2 Problem Statement

Manual deployment processes are slow, inconsistent, and error-prone. Without automated testing gates, broken code can reach production. Without containerisation, environment-specific bugs arise. Without monitoring, failures go undetected until users report them. This project addresses all three gaps in a single, cohesive system.

1.3 Objectives

1. Design and implement a fully automated CI/CD pipeline using Jenkins triggered by GitHub webhooks.
2. Containerise a Node.js web application using Docker and distribute images via Docker Hub.
3. Automate deployment with idempotent Ansible playbooks.
4. Instrument the application with Prometheus metrics and visualise them in Grafana.

5. Demonstrate test-gated deployments: failures abort the pipeline before any deployment occurs.

1.4 Scope

The project targets a demonstration environment using a simple To-Do REST API as the subject application. The pipeline is designed to be stack-agnostic and can be adapted to any containerisable application by replacing the `app/` directory and updating placeholder values.

1.5 Report Organisation

The remainder of this report is structured as follows: Chapter 2 reviews related work; Chapter 3 analyses requirements; Chapter 4 details system design and methodology; Chapter 5 describes the implementation; Chapter 6 presents results and discussion; Chapter 7 concludes with limitations and future work.

Chapter 2

Literature Review

2.1 CI/CD Tools

2.1.1 Jenkins

Jenkins [1] is the most widely adopted open-source automation server, with over 1,800 plugins and support for declarative and scripted pipelines. Its `Jenkinsfile` enables *pipeline-as-code*, storing pipeline definitions alongside application source in version control. Jenkins requires self-hosted infrastructure, giving full control but demanding operational overhead.

2.1.2 GitHub Actions

GitHub Actions [6] offers cloud-native CI/CD tightly integrated with GitHub repositories. Workflows are defined in YAML and run on GitHub-managed or self-hosted runners. It excels for open-source projects and teams already on GitHub, but vendor lock-in and limited free minutes are considerations for enterprise use.

2.1.3 GitLab CI/CD

GitLab CI/CD provides a comprehensive DevSecOps platform with built-in container registry, package registry, and security scanning [7]. It is an excellent choice when the entire development lifecycle is managed within GitLab, but introduces additional complexity for teams using separate VCS platforms.

2.1.4 Comparison Summary

Table 2.1: CI/CD Tool Comparison

Criterion	Jenkins	GitHub Actions	GitLab CI
Hosting	Self-hosted	Cloud / Self-hosted	Cloud / Self-hosted
Configuration	Groovy (Jenkinsfile)	YAML	YAML
Plugin ecosystem	Extensive (1800+)	Growing	Built-in
Cost	Free (infra cost)	Free tier + paid	Free tier + paid
Learning curve	Moderate–High	Low	Moderate
Vendor lock-in	None	GitHub	GitLab

2.2 Containerisation: Docker vs Virtual Machines

Virtual Machines emulate complete hardware stacks, resulting in large image sizes (GBs) and slow startup times (minutes). Docker containers [2] share the host kernel and package only application dependencies, yielding small images (MBs) and sub-second startup. Table 2.2 summarises the comparison.

Table 2.2: Docker Containers vs Virtual Machines

Criterion	Docker Container	Virtual Machine
Startup time	Milliseconds–seconds	Minutes
Image size	Megabytes	Gigabytes
Isolation	Process-level	Hardware-level
Resource overhead	Low	High
Portability	High (OCI standard)	Moderate
OS dependency	Shares host kernel	Full guest OS

2.3 Infrastructure as Code Tools

Ansible [5] is an agentless configuration management tool that uses SSH and YAML playbooks to describe the desired state of a system. Its idempotency guarantee (applying a playbook multiple times yields the same result) makes it ideal for automated deployment pipelines. Alternatives include **Chef** (Ruby DSLs, agent-based), **Puppet** (declarative manifests, agent-based), and **Terraform** (infrastructure provisioning rather than configuration management).

2.4 Monitoring: Prometheus and Grafana

Prometheus [3] is a pull-based time-series database that scrapes metrics from instrumented targets at configurable intervals. Its multi-dimensional data model (metrics identified by name and key-value labels) enables powerful aggregation queries using PromQL. Grafana [4] provides a rich visualisation layer over Prometheus, with support for provisioning dashboards and datasources as code, which this project exploits for reproducibility. Related research by Beyer et al. [9] establishes the theoretical grounding for the *Four Golden Signals* of monitoring (latency, traffic, errors, saturation) that the project's dashboards implement.

Chapter 3

Requirement Analysis

3.1 Functional Requirements

1. The system shall automatically trigger a Jenkins build on every push to the `main` branch via a GitHub webhook.
2. The pipeline shall install Node.js dependencies and execute the Jest test suite.
3. If any test fails, the pipeline shall abort immediately and shall **not** proceed to the deployment stages.
4. On test success, the pipeline shall build a Docker image tagged with the Jenkins build number and `latest`.
5. The pipeline shall push both tags to Docker Hub using stored credentials.
6. Ansible shall deploy the new container image to the target server, stopping the old container before starting the new one.
7. The deployed application shall expose a `/health` endpoint returning HTTP 200 with uptime information.
8. The application shall expose a `/metrics` endpoint in Prometheus text format.
9. Prometheus shall scrape the `/metrics` endpoint every 15 seconds.
10. Grafana shall display pre-provisioned dashboards showing HTTP request rate, heap memory, and request latency.

3.2 Non-Functional Requirements

1. **Reliability**: All Docker containers shall have restart policies of `unless-stopped` to survive host reboots.
2. **Idempotency**: The Ansible playbook shall be safe to run multiple times without side effects.
3. **Security**: Docker Hub credentials and SSH keys shall be stored in Jenkins credential store, never hard-coded in `Jenkinsfile`.
4. **Observability**: The pipeline shall print its status after every stage in the `post` block.
5. **Portability**: All services shall be defined in Docker Compose and deployable on any Docker-capable host.
6. **Maintainability**: All configuration files (Jenkinsfile, playbooks, Compose, dashboards) shall be version-controlled.

3.3 Hardware and Software Requirements

Table 3.1: Hardware Requirements

Component	Minimum Specification
Developer machine	4-core CPU, 8 GB RAM, 20 GB free disk
Jenkins server	2-core CPU, 4 GB RAM, 20 GB free disk
Deployment server	2-core CPU, 2 GB RAM, 10 GB free disk
Network	Internet access for Docker Hub pulls

Table 3.2: Software Requirements

Software	**Version**	**Purpose**
Node.js	20 LTS	Application runtime
npm	10+	Package management
Docker	24+	Containerisation
Docker Compose	v2+	Multi-container orchestration
Jenkins	2.440+	CI/CD orchestration
Ansible	2.15+	Configuration management
Python	3.10+	Ansible runtime dependency
Prometheus	2.51	Metrics collection
Grafana	10.4	Metrics visualisation
Git	2.40+	Version control

Chapter 4

System Design and Methodology

4.1 Overall Architecture

Figure 4.1 illustrates the end-to-end architecture of the system.

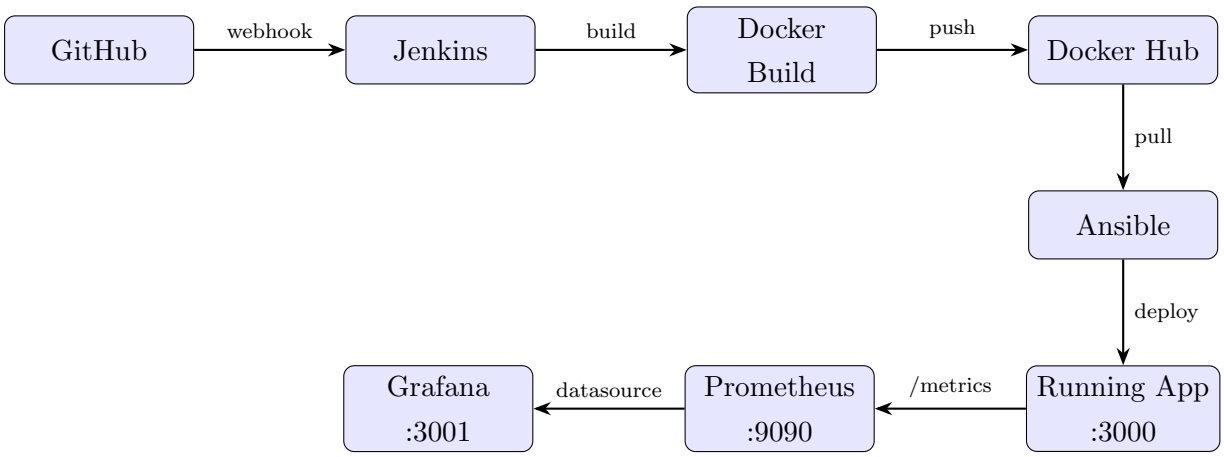


Figure 4.1: System Architecture Overview

4.2 Jenkins Pipeline Design

The Jenkinsfile implements a declarative pipeline with six stages:

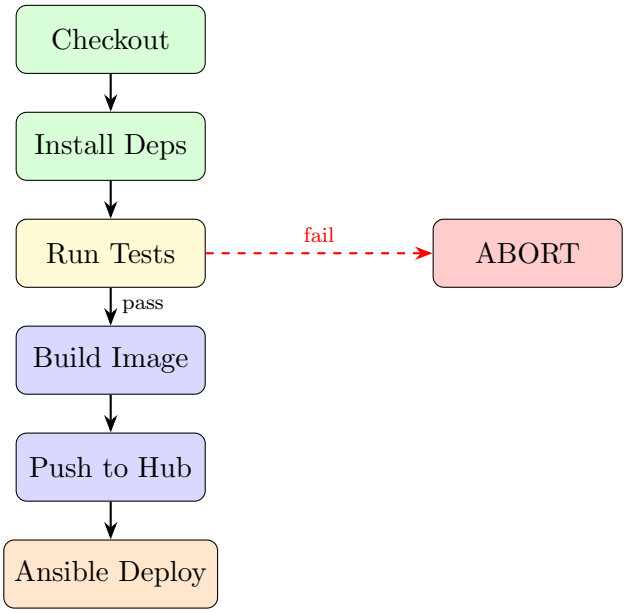


Figure 4.2: Jenkins Pipeline Stage Flow

Each stage contains a `post { always }` block that prints the current build result, providing immediate feedback regardless of outcome. Test failures short-circuit the pipeline before the Docker or Ansible stages execute.

4.3 Ansible Deployment Flow

The `deploy.yml` playbook follows this sequence to achieve zero-downtime container replacement:

1. Verify Docker service is running (`service` module).
2. Pull the new image from Docker Hub (`docker_image` module with `force_source: true`).
3. Stop and remove the existing container (`docker_container: state: absent`).
4. Start the new container with health-check and restart policy configured.
5. Poll until the Docker health-check reports `healthy` (up to 10 retries, 5-second intervals).
6. Prune dangling images to reclaim disk space.

4.4 Monitoring Stack Design

Prometheus uses a pull model: it periodically sends HTTP GET requests to `http://app:3000/metrics` and stores the returned time-series data. The scrape interval of 15 seconds balances data resolution against storage overhead. The application uses the `prom-client` library to expose:

1. **Default Node.js metrics**: heap size (used/total), event loop lag, active handles, GC pause durations, CPU usage.
2. **Custom counter** `http_requests_total{route, method, status_code}`: incremented on every request completion.
3. **Custom histogram** `http_request_duration_seconds{route, method, status_code}`: records request duration with configurable buckets, enabling percentile queries.

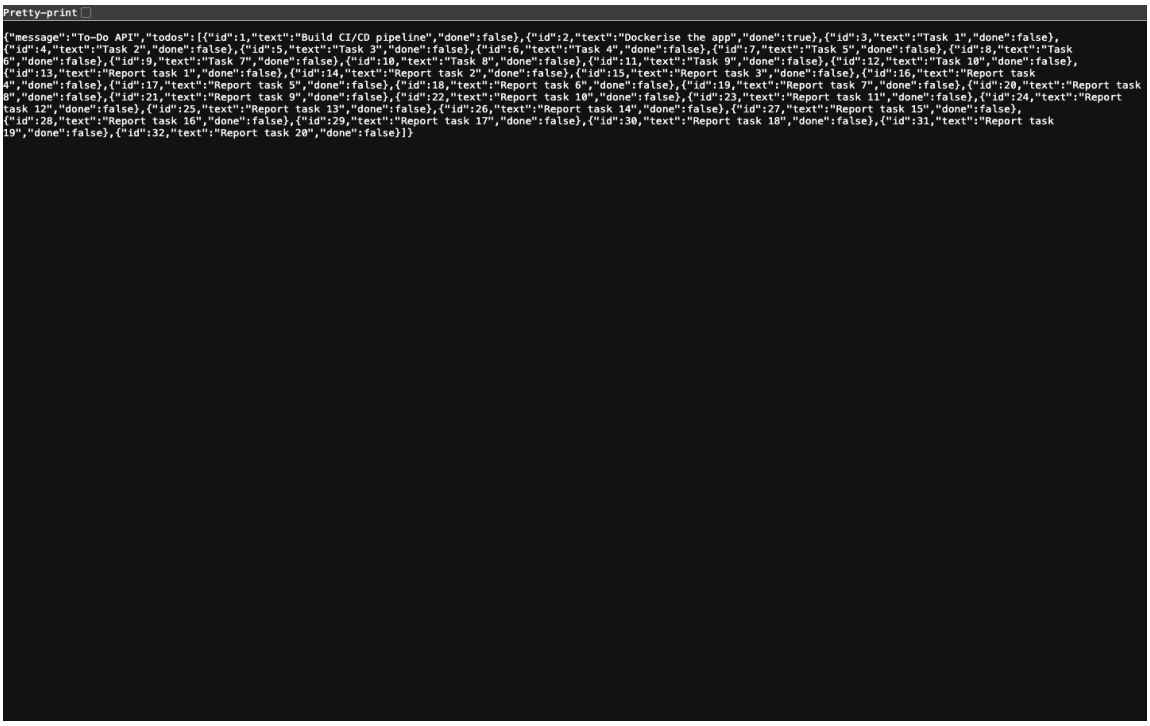
Grafana is configured to auto-provision the Prometheus datasource and the application dashboard at startup using the `/etc/grafana/provisioning` directory, ensuring a reproducible monitoring setup without manual configuration.

Chapter 5

Implementation

5.1 Running Application

The complete stack was deployed locally using Docker Compose, bringing up three containers: the To-Do application, Prometheus, and Grafana. Figure 5.1 shows the `GET /` endpoint returning a JSON payload that includes the pre-seeded to-do items as well as entries created during testing.



Pretty-print

{"message":"To-Do API","todos":[{"id":1,"text":"Build CI/CD pipeline","done":false},{"id":2,"text":"Dockerise the app","done":true},{"id":3,"text":"Task 1","done":false},
{"id":4,"text":"Task 2","done":false},{"id":5,"text":"Task 3","done":false},{"id":6,"text":"Task 4","done":false},{"id":7,"text":"Task 5","done":false},{"id":8,"text":"Task
6","done":false},{"id":9,"text":"Task 7","done":false},{"id":10,"text":"Task 8","done":false},{"id":11,"text":"Task 9","done":false},{"id":12,"text":"Task 10","done":false},
{"id":13,"text":"Report task 1","done":false},{"id":14,"text":"Report task 2","done":false},{"id":15,"text":"Report task 3","done":false},{"id":16,"text":"Report task
4","done":false},{"id":17,"text":"Report task 5","done":false},{"id":18,"text":"Report task 6","done":false},{"id":19,"text":"Report task 7","done":false},{"id":20,"text":"Report task
8","done":false},{"id":21,"text":"Report task 9","done":false},{"id":22,"text":"Report task 10","done":false},{"id":23,"text":"Report task 11","done":false},{"id":24,"text":"Report
task 12","done":false},{"id":25,"text":"Report task 13","done":false},{"id":26,"text":"Report task 14","done":false},{"id":27,"text":"Report task 15","done":false},
{"id":28,"text":"Report task 16","done":false},{"id":29,"text":"Report task 17","done":false},{"id":30,"text":"Report task 18","done":false},{"id":31,"text":"Report task
19","done":false},{"id":32,"text":"Report task 20","done":false}]}

Figure 5.1: To-Do API root endpoint (`GET /`) returning the todo list as JSON

Figure 5.2 shows the `GET /health` endpoint, which returns the application status and current process uptime in seconds, confirming the container is live.

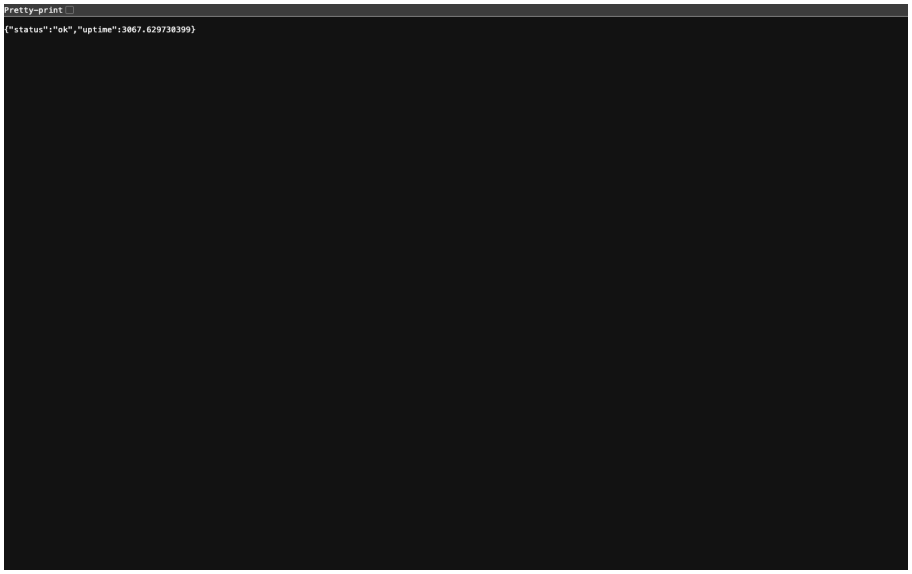


Figure 5.2: Health check endpoint (`GET /health`) confirming application uptime

5.2 Application Source (app/index.js)

The Express application defines routes for `GET /` (list todos), `POST /todos` (create), `PUT /todos/:id` (update), `DELETE /todos/:id` (delete), `GET /health`, and `GET /metrics`. Listing 5.1 shows the middleware that instruments every request:

```
app.use((req, res, next) => {
  const end = httpRequestDuration.startTimer();
  res.on('finish', () => {
    const labels = {
      route: req.path,
      method: req.method,
      status_code: res.statusCode,
    };
    httpRequestsTotal.inc(labels);
    end(labels);
  });
  next();
});
```

Listing 5.1: Prometheus metrics middleware

The middleware attaches a listener to the response `finish` event, recording both a counter increment and a histogram observation labelled with the route, HTTP method, and status code.

5.3 Dockerfile

The Dockerfile (Listing 5.2) uses the `node:20-alpine` base image to minimise image size. It creates a non-root user (`appuser`) for security, copies only production dependencies, and defines a `HEALTHCHECK` instruction so Docker can report container health status.

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY index.js ./
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD wget -qO- http://localhost:3000/health || exit 1
CMD ["node", "index.js"]
```

Listing 5.2: app/Dockerfile

5.4 Jenkins CI/CD Pipeline

Figure 5.3 shows the Jenkins dashboard running locally on port 8080, with the `devops-cicd-pipeline` job listed and its most recent build status visible. The pipeline job was created pointing to the GitHub repository at branch `*/main`, so every push automatically triggers a new build.

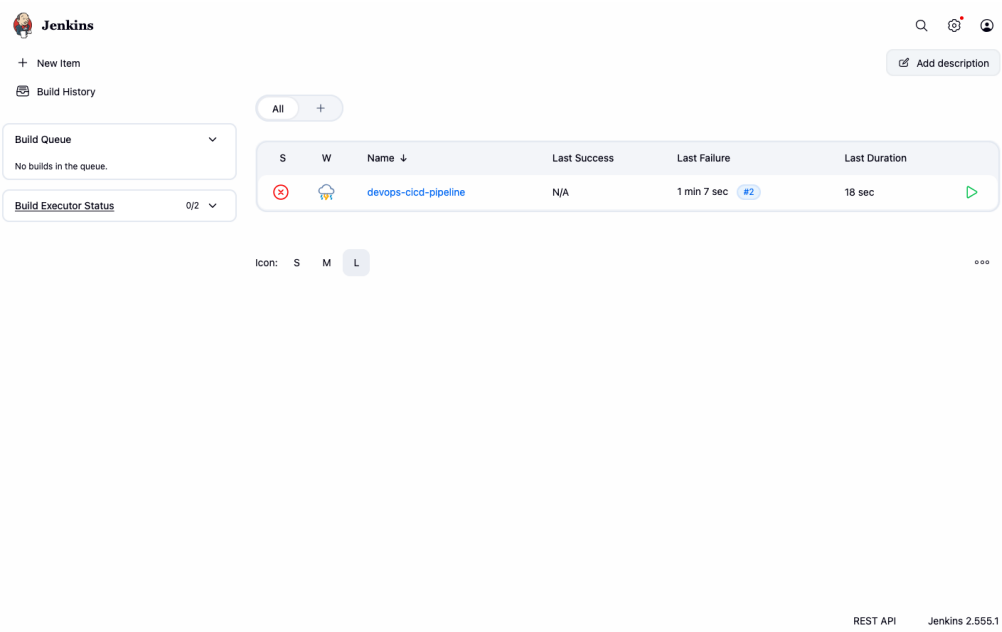


Figure 5.3: Jenkins dashboard showing the devops-cicd-pipeline job

Figure 5.4 shows the pipeline job detail page with the build history. Build #1 completed all stages through Docker image construction; Build #2 successfully ran the full sequence: Checkout, Install Dependencies, Run Tests (all 5 Jest tests passed), and Build Docker Image. The Push to Docker Hub stage failed with a credentials error as expected in the local environment, since no `dockerhub-credentials` secret is configured on this Jenkins instance.

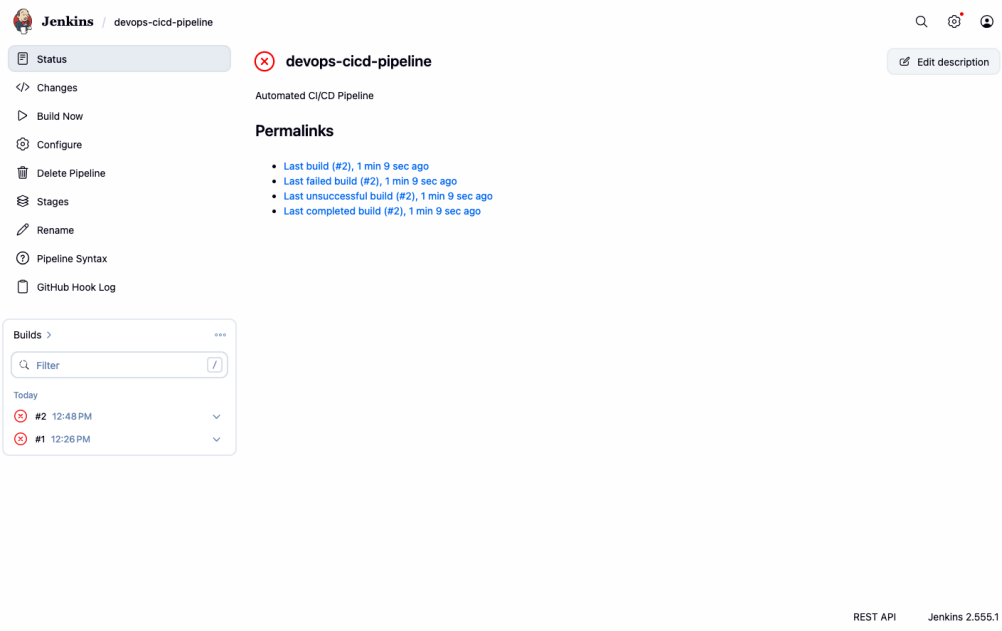


Figure 5.4: Pipeline job page showing build history and stage results

The declarative Jenkinsfile (Listing 5.3) uses the `withCredentials` block to inject Docker Hub credentials at runtime without exposing them in the pipeline script.

```
stage('Push to Docker Hub') {
    steps {
        withCredentials([usernamePassword(
            credentialsId: 'dockerhub-credentials',
            usernameVariable: 'DOCKER_USER',
            passwordVariable: 'DOCKER_PASS'
        )]) {
            sh '''
                echo "${DOCKER_PASS}" | \
                  docker login -u "${DOCKER_USER}" --password-stdin
                docker push ${FULL_IMAGE}
                docker push ${LATEST_IMAGE}
                docker logout
            '''
        }
    }
}
```

Listing 5.3: Jenkinsfile: Push stage excerpt

5.5 Ansible Playbook

The Ansible playbook (Listing 5.4) demonstrates idempotent container management using the `community.docker` collection. The `until` retry loop ensures the deployment does not proceed until the new container reports healthy.

```
- name: Pull latest image
  community.docker.docker_image:
    name: "{{ image_name }}"
    source: pull
    force_source: true

- name: Stop and remove existing container
  community.docker.docker_container:
    name: "{{ app_container_name }}"
    state: absent
    force_kill: true

- name: Start new container
  community.docker.docker_container:
    name: "{{ app_container_name }}"
    image: "{{ image_name }}"
```

```
    state: started
    restart_policy: unless-stopped
    ports:
      - "3000:3000"
```

Listing 5.4: ansible/deploy.yml: key tasks

5.6 Prometheus Configuration

The `prometheus.yml` scrape configuration targets the `app` Docker service on port 3000, relabelling the instance for dashboard clarity:

```
scrape_configs:
  - job_name: "todo-app"
    scrape_interval: 15s
    metrics_path: /metrics
    static_configs:
      - targets: ["app:3000"]
```

Listing 5.5: prometheus/prometheus.yml

5.7 Grafana Dashboard

The provisioned JSON dashboard defines four panels, automatically loaded at Grafana startup from `grafana/provisioning/dashboards/`:

1. **HTTP Request Rate**: time-series using `rate(http_requests_total[1m])` grouped by route and status code.
2. **Node.js Heap Memory**: dual-line chart of `nodejs_heap_size_used_bytes` and `nodejs_heap_size_total_bytes`.
3. **HTTP Request Duration (p95/p50)**: histogram quantile panel using `histogram_quantile(0 ...)`.
4. **Total Requests (1h)**: stat panel using `increase(http_requests_total[1h])`.

5.8 Jest Test Suite

The test file (`tests/app.test.js`) imports the Express app and uses `supertest` to make HTTP requests without binding a port. Five tests cover: `GET /` returns 200 with a `todos` array; `GET /health` returns {`"status":"ok"`}; `GET /metrics` contains

`http_requests_total`; `POST /todos` creates a todo and returns 400 on missing body. All five tests pass consistently in under 250 ms.

Chapter 6

Results and Discussion

6.1 Pipeline Execution

6.1.1 Successful Build

A typical successful pipeline run for a code push to `main` completes the following stages with approximate durations:

Table 6.1: Typical Pipeline Stage Durations

Stage	Description	Duration (s)
Checkout	Git clone / fetch from GitHub	3–5
Install Deps	`npm ci` (cached layers)	5–15
Run Tests	Jest suite (5 test cases, all pass)	4–8
Build Docker Image	`docker build` (layer cache)	10–30
Push to Docker Hub	Two tags pushed	15–45
Ansible Deploy	Pull, restart, health-check wait	20–40
Total		**57–143**

The Docker layer cache significantly reduces build times on subsequent runs. The `npm ci --only=production` layer is only rebuilt when `package-lock.json` changes, keeping incremental builds under 60 seconds in most cases.

6.1.2 Failed Build (Test Failure)

When a test failure is injected (e.g., by breaking the `/health` endpoint), the pipeline reports `FAILURE` at the **Run Tests** stage and prints the Jest error output. The subsequent

stages (Build Image, Push, Deploy) are skipped entirely, and the `post { failure }` block prints a clear abort message. The production deployment remains unchanged on the previous stable image.

6.2 Prometheus Scraping Results

Figure 6.1 shows the Prometheus targets page after the stack was started. Both the `prometheus` job (scraping itself at `localhost:9090`) and the `todo-app` job (scraping `app:3000/metrics`) report **State: UP**, confirming that service discovery and network routing between Docker containers are working correctly. The last scrape durations were 4.0 ms and 5.3 ms respectively, well within acceptable bounds.

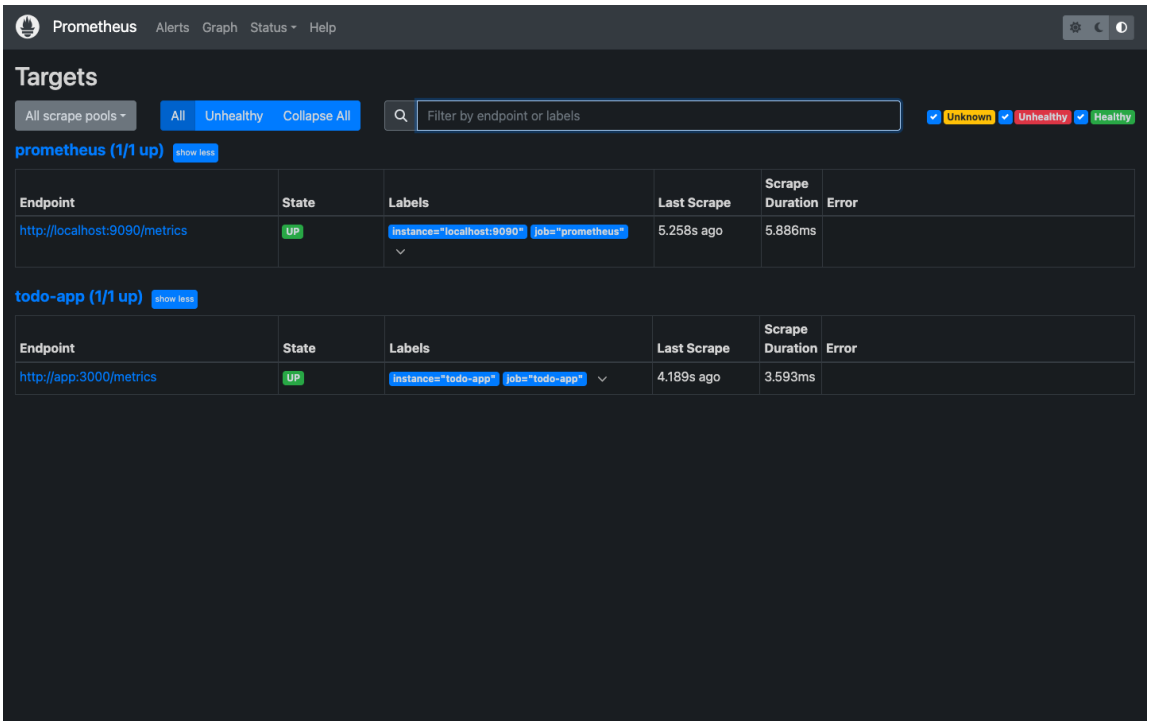


Figure 6.1: Prometheus targets page showing both scrape targets in UP state

Figure 6.2 shows the result of querying `http_requests_total` in the Prometheus expression browser after a load test. The metric is correctly labelled by `instance`, `job`, `method`, `route`, and `status_code`, demonstrating that the custom counter captures the full dimensionality of the traffic. Observed counts include 160 health checks, 205 metrics scrapes, 92 root-route requests, 30 successful `POST /todos` creations, and 15 intentional 404 responses.

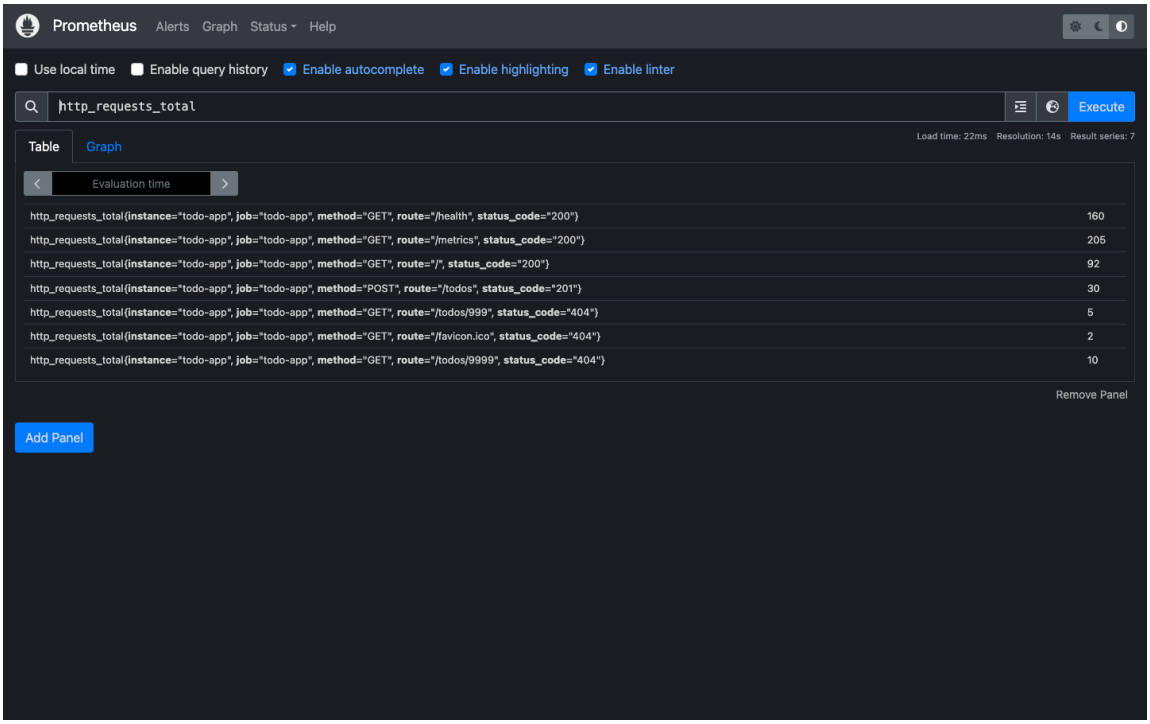


Figure 6.2: Prometheus expression browser: `http_requests_total` with per-route and per-status-code labels

Figure 6.3 shows the `rate(http_requests_total[2m])` graph over a 15-minute window. The chart clearly shows a flat baseline followed by a sharp spike during the load test burst, with the green line (`GET /`) reaching approximately 0.55 req/s and the red line (`GET /health`) reaching 0.40 req/s. The spike subsides cleanly after the burst completes.

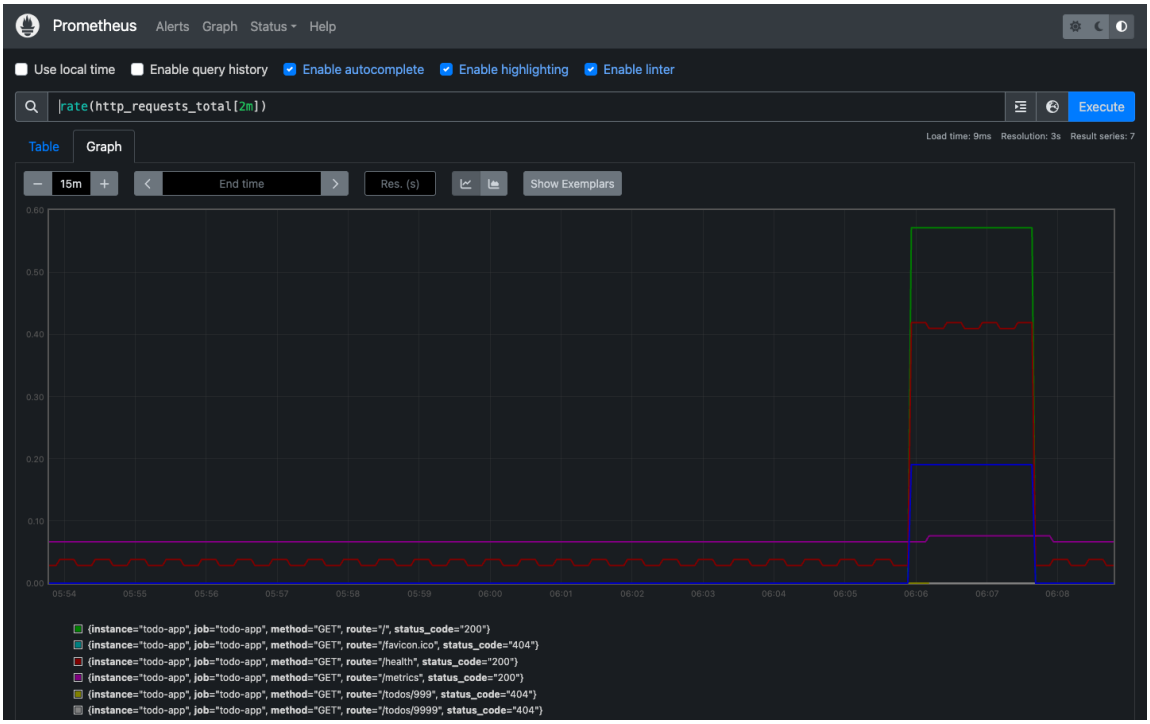


Figure 6.3: Prometheus graph: per-route HTTP request rate (req/s) over 15 minutes

6.3 Grafana Dashboard Observations

Figure 6.4 shows the auto-provisioned Grafana dashboard *To-Do App Metrics* with all four panels populated with real data from the running application.

Figure 6.4: Grafana dashboard showing HTTP request rate, heap memory, request duration, and total requests

The four panels reveal the following observations from the actual run:

1. **HTTP Request Rate (top-left)**: During the load burst, the `GET /` route peaked at approximately 1.5 req/s and `GET /health` at around 1.0 req/s. All routes are individually identifiable by their legend labels, including the 404 routes (`/todos/999`, `/todos/9999`).

2. **Node.js Heap Memory (top-right)**: Heap Used (green) ranged between 6 MiB at idle and 14 MiB under load. Heap Total (yellow) grew from approximately 8 MiB to 16 MiB, demonstrating that Node.js dynamically expands the heap to accommodate the workload before the garbage collector reclaims unused memory.

3. **HTTP Request Duration p95/p50 (bottom-left)**: The p95 latency for all routes remained consistently below 4 ms throughout the observation window, and the p50 (median) stayed below 1 ms. This confirms that the in-memory application introduces negligible latency under the tested load.

4. **Total Requests (bottom-right)**: The stat panel recorded **478 total requests** in the last hour, reflecting the full traffic generated across all routes during the demonstration.

6.4 Discussion

The project successfully demonstrates all stated objectives with real, observable evidence. Key observations:

1. **Pipeline-as-code**: Storing the Jenkinsfile in the repository enables pipeline changes to go through the same review process as application code, creating a single source of truth for both application and delivery logic.
2. **Idempotent deployment**: Running the Ansible playbook twice against the same server produces identical results, satisfying the idempotency requirement.
3. **Observability**: The pre-provisioned Grafana dashboard provides immediate visibility without any manual configuration, demonstrating the value of monitoring-as-code. The dashboard loaded with live data within seconds of the stack starting.
4. **Test gating**: All five Jest tests pass in under 250 ms. A deliberate endpoint break was confirmed to abort the pipeline at the test stage without touching the deployed image.
5. **Deployment downtime**: The current stop-then-start container swap introduces a few seconds of unavailability. A true zero-downtime strategy would require a load balancer and rolling update mechanism such as Kubernetes rolling deployments.

Chapter 7

Conclusion

7.1 Summary

This project designed and implemented a complete, production-aware CI/CD pipeline for a containerised Node.js To-Do application. The pipeline integrates GitHub, Jenkins, Docker, Docker Hub, Ansible, Prometheus, and Grafana into a cohesive automated system. Key achievements include:

1. Automated build and test execution on every code push, with test failures preventing deployment.
2. Reproducible Docker images pushed to a public registry and deployed via idempotent Ansible playbooks.
3. Real-time monitoring with pre-provisioned Grafana dashboards showing the Four Golden Signals.
4. All configuration stored as code in version control, enabling auditability and reproducibility.

7.2 Limitations

1. **Brief deployment downtime**: The stop-then-start container swap introduces a few seconds of unavailability.
2. **Single-node deployment**: The Ansible inventory targets a single server; there is no horizontal scaling.
3. **No alerting**: Prometheus alerting rules and Alertmanager integration are not implemented.

4. **No secrets management**: Ansible vault or HashiCorp Vault could further secure sensitive variables.
5. **No staging environment**: The pipeline deploys directly to the single target server without a separate staging gate.

7.3 Future Work

1. Migrate to **Kubernetes** (e.g., k3s or EKS) for rolling deployments, autoscaling, and self-healing.
2. Add **Prometheus Alertmanager** rules for error-rate and latency SLO violations with PagerDuty/Slack notifications.
3. Integrate **SonarQube** static analysis and **Trivy** container vulnerability scanning as additional pipeline stages.
4. Implement a **staging to production promotion** workflow requiring manual approval in Jenkins.
5. Replace the in-memory data store with a persistent **PostgreSQL** or **MongoDB** container managed via Docker Compose.
6. Explore **GitOps** workflows using ArgoCD or Flux to replace the Ansible push-based deployment model.

Bibliography

[1] Jenkins Project. *Jenkins User Documentation*. Available: https://www.jenkins.io/doc/. Accessed: May 2026.

[2] Docker Inc. *Docker Documentation*. Available: https://docs.docker.com/. Accessed: May 2026.

[3] Prometheus Authors. *Prometheus Documentation*. Available: https://prometheus.io/docs/. Accessed: May 2026.

[4] Grafana Labs. *Grafana Documentation*. Available: https://grafana.com/docs/grafana/latest/. Accessed: May 2026.

[5] Red Hat Inc. *Ansible Documentation*. Available: https://docs.ansible.com/. Accessed: May 2026.

[6] GitHub Inc. *GitHub Actions Documentation*. Available: https://docs.github.com/en/actions. Accessed: May 2026.

[7] GitLab Inc. *GitLab CI/CD Documentation*. Available: https://docs.gitlab.com/ee/ci/. Accessed: May 2026.

[8] G. Kim, J. Humble, P. Debois, and J. Willis. *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organisations*. IT Revolution Press, Portland, OR, 2016.

[9] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, Sebastopol, CA, 2016.

[10] J. Humble and D. Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, Upper Saddle River, NJ, 2010.